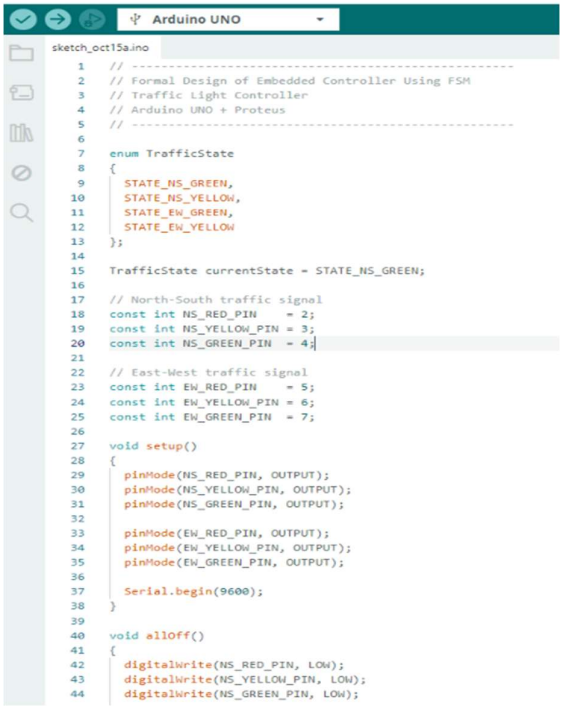
ARDUINO CODE




```
// ------------------------------------------------------
// Formal Design of Embedded Controller Using FSM
// Traffic Light Controller
// Arduino UNO + Proteus
// ------------------------------------------------------

enum TrafficState
{
  STATE_NS_GREEN,
  STATE_NS_YELLOW,
  STATE_EW_GREEN,
  STATE_EW_YELLOW
};

TrafficState currentState = STATE_NS_GREEN;

// North-South traffic signal
const int NS_RED_PIN    = 2;
const int NS_YELLOW_PIN = 3;
const int NS_GREEN_PIN  = 4;

// East-West traffic signal
const int EW_RED_PIN    = 5;
const int EW_YELLOW_PIN = 6;
const int EW_GREEN_PIN  = 7;

void setup()
{
  pinMode(NS_RED_PIN, OUTPUT);
  pinMode(NS_YELLOW_PIN, OUTPUT);
  pinMode(NS_GREEN_PIN, OUTPUT);

  pinMode(EW_RED_PIN, OUTPUT);
  pinMode(EW_YELLOW_PIN, OUTPUT);
  pinMode(EW_GREEN_PIN, OUTPUT);

  Serial.begin(9600);
}

void allOff()
{
  digitalWrite(NS_RED_PIN, LOW);
  digitalWrite(NS_YELLOW_PIN, LOW);
  digitalWrite(NS_GREEN_PIN, LOW);
```

```
digitalWrite(EW_RED_PIN, LOW);
digitalWrite(EW_YELLOW_PIN, LOW);
digitalWrite(EW_GREEN_PIN, LOW);
```

```
oid enterState(TrafficState state)

allOff();

switch (state)
{
  case STATE_NS_GREEN:

    digitalWrite(NS_GREEN_PIN, HIGH);
    digitalWrite(EW_RED_PIN, HIGH);

    Serial.println("STATE S0: NS GREEN - EW RED");

    break;

  case STATE_NS_YELLOW:

    digitalWrite(NS_YELLOW_PIN, HIGH);
    digitalWrite(EW_RED_PIN, HIGH);

    Serial.println("STATE S1: NS YELLOW - EW RED");

    break;

  case STATE_EW_GREEN:

    digitalWrite(NS_RED_PIN, HIGH);
    digitalWrite(EW_GREEN_PIN, HIGH);

    Serial.println("STATE S2: NS RED - EW GREEN");

    break;

  case STATE_EW_YELLOW:

    digitalWrite(NS_RED_PIN, HIGH);
    digitalWrite(EW_YELLOW_PIN, HIGH);

    Serial.println("STATE S3: NS RED - EW YELLOW");
```

```
    Serial.println("STATE S3: NS RED - EW YELLOW");

    break;
  }
}

void loop()
{
  switch (currentState)
  {
    case STATE_NS_GREEN:

      enterState(STATE_NS_GREEN);
      delay(10000);
      currentState = STATE_NS_YELLOW;

      break;

    case STATE_NS_YELLOW:

      enterState(STATE_NS_YELLOW);
      delay(3000);
      currentState = STATE_EW_GREEN;

      break;

    case STATE_EW_GREEN:

      enterState(STATE_EW_GREEN);
      delay(10000);
      currentState = STATE_EW_YELLOW;

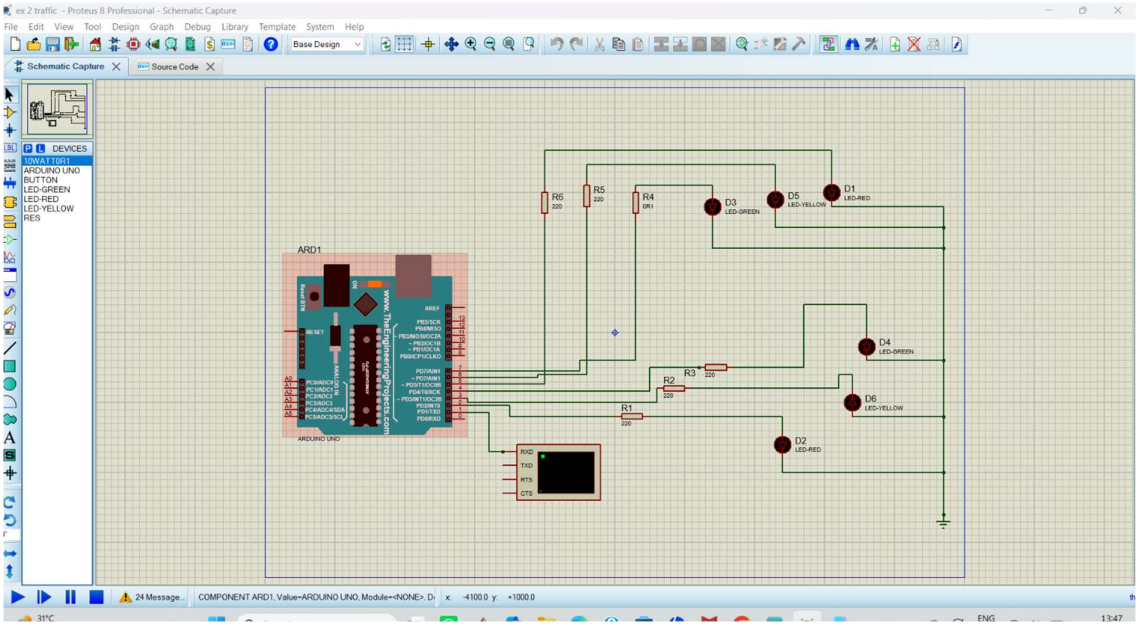
      break;

    case STATE_EW_YELLOW:

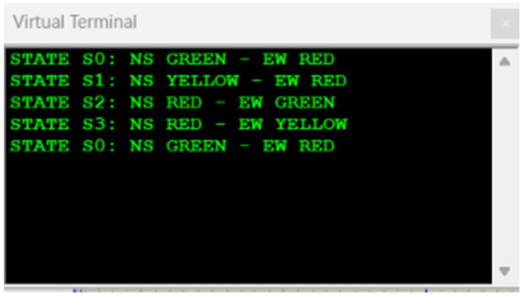
      enterState(STATE_EW_YELLOW);
      delay(3000);
      currentState = STATE_NS_GREEN;

      break;
  }
}
```

Input



output


Virtual Terminal
STATE S0: NS GREEN - EW RED
STATE S1: NS YELLOW - EW RED
STATE S2: NS RED - EW GREEN
STATE S3: NS RED - EW YELLOW
STATE S0: NS GREEN - EW RED